# APPLICATION LAYER PROTOCOLS (EMAIL & DNS)

**CIS*3210**

**Fall 2026**

**Sajid Marhon**

Based on notes for Computer Networking: A Top-Down Approach, 9th ed., by J. Kurose and K. Ross

# DNS: Domain Name System



- People have multiple identifiers: e.g., SSN, name, passport number
- Internet devices (hosts/routers) also have identifiers:
  - IPv4 address (32-bit): used for routing and addressing datagrams
  - Domain name (e.g., cs.umass.edu): human-friendly identifier
- Question: how do we translate between IP addresses and domain names (both directions)?
- **DNS (Domain Name System):** a distributed, hierarchical database of name servers
- **Application-layer protocol**: enables hosts and DNS servers to exchange messages for name/address translation
  - A core Internet service, yet implemented at the application layer
  - Keeps most complexity at the network edge, not in the core

# DNS Services



- **DNS role**:
  - Translates hostnames ↔ IP addresses
  - Implemented as a distributed hierarchical database of DNS servers
  - An application-layer protocol (over UDP, port 53) to query that database
- **Usage**: relied on by many protocols (HTTP, SMTP) for hostname resolution
- **Example flow**:
  1. Browser extracts hostname from URL
  2. DNS client queries a DNS server
  3. DNS reply returns the IP address
  4. Browser uses IP + port (e.g., 80) to open a TCP connection to the server
- DNS adds delay, but caching in nearby DNS servers reduces latency and traffic

# DNS Services



- **Additional DNS services:**
  - **Host aliasing**: map simple names to canonical hostnames

    Example: www.example.com
  - **Mail server aliasing**: map easy-to-remember email domains (e.g., yahoo.com) to canonical mail server names via MX (mail exchange) records
  - **Load distribution**: multiple IPs for one hostname; DNS rotates returned addresses to balance load across replicated servers

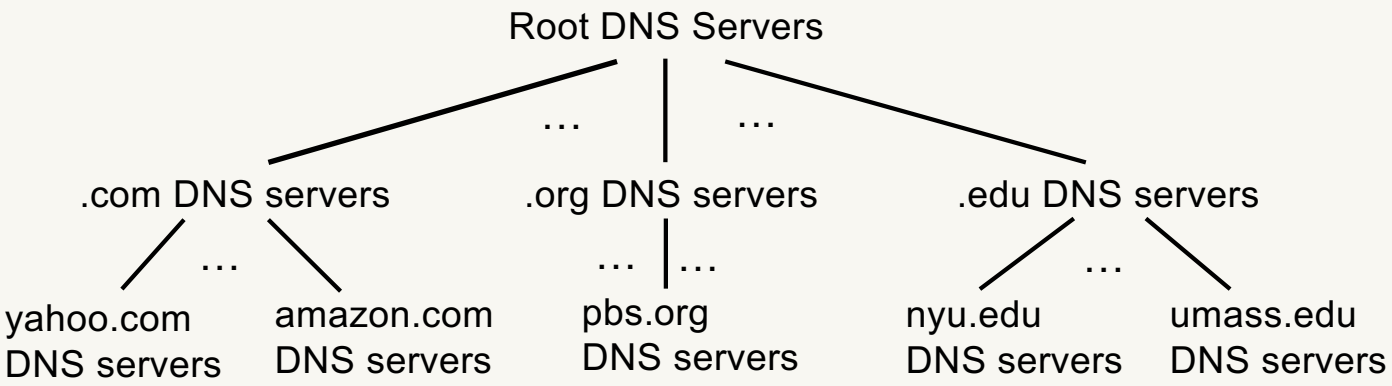
# DNS Structure



- **Why DNS isn't centralized:**
  - Would create a single point of failure
  - Couldn't handle the enormous query traffic volume
  - A remote central database would add high latency for many users
  - Difficult to maintain and update at Internet scale
  - Comcast DNS servers alone: 600B DNS queries/day
  - Akamai DNS servers alone: 2.2T DNS queries/day

# DNS: Distributed, Hierarchical






*Root*

*Top Level Domain*

*Authoritative*

**Client wants IP address for www.amazon.com; 1st approximation:**

1. client queries root server to find .com DNS server
2. client queries .com DNS server to get amazon.com DNS server
3. client queries amazon.com DNS server to get IP address for www.amazon.com

# DNS: Distributed, Hierarchical



- **Root DNS servers**
  - About 2000 instances worldwide, handling tens of billions of queries daily
  - Copies of a small set of 13 root servers, run by multiple organizations under IANA coordination
  - Main role: provide IP addresses of TLD servers




- **Top-Level Domain (TLD) servers**
  - One (or cluster) for each TLD (e.g., .com, .org, .net, .edu, .gov, .uk, .fr, .ca, .jp)
  - Example maintainers: Verisign for .com, Educause for .edu
  - Infrastructure is large/complex; provides IP addresses of authoritative DNS servers

# Local DNS Name Servers



- **Local DNS server:**
  - Not part of hierarchy but central to DNS operation
  - Provided by ISPs (via **DHCP**) or can use public ones (e.g., **Google DNS**)
  - Typically located close to the host (same LAN or a few routers away)
- **Role of local DNS server:**
  - Acts as a proxy for host queries
  - Forwards requests into DNS hierarchy and returns results to host
  - each ISP has local DNS name server; to find yours:
    - macOS: % scutil --dns
    - Windows: >ipconfig /all



## Discussion Question:

**Why is a local DNS server operationally central even though it is not one of the root, TLD, or authoritative levels?**

A local DNS server is central to the lookup process because it does the work on behalf of the client.

The host normally sends its query to the local resolver, which performs or forwards the remaining resolution and caches results. It is therefore the client's gateway to the hierarchy without being a delegated hierarchy level.

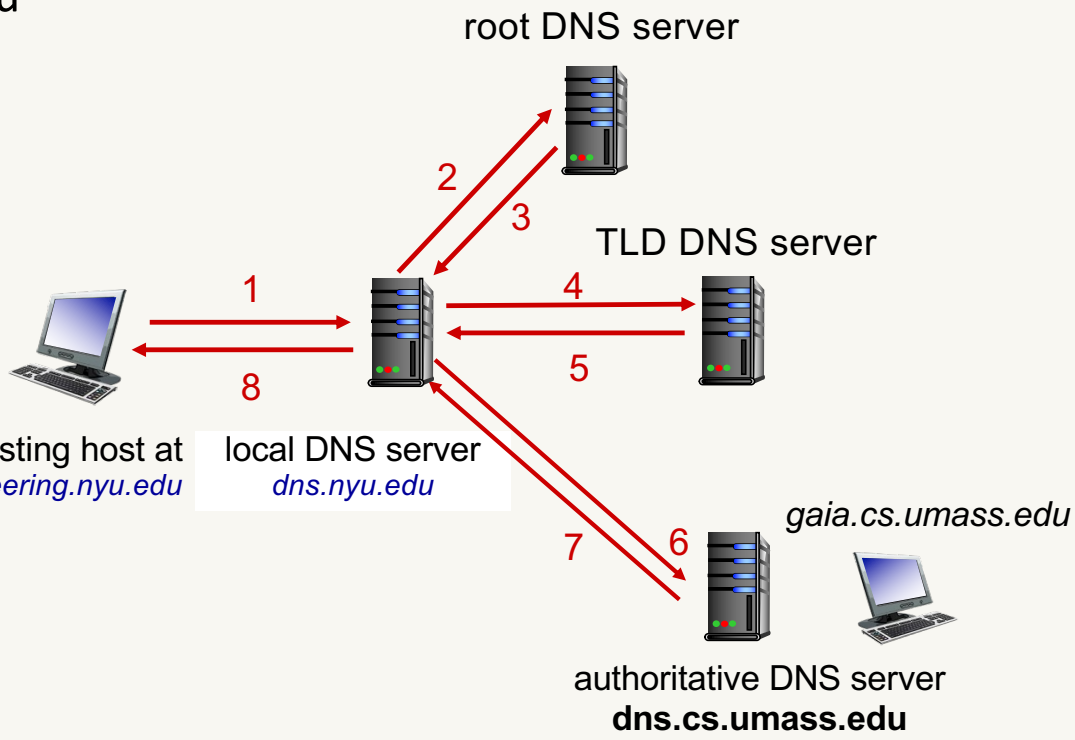
# DNS Name Resolution: Iterative Query



**Example:** host at engineering.nyu.edu wants IP address for gaia.cs.umass.edu

## Iterative query:

- contacted server replies with name of server to contact
- "I don't know this name, but ask this server"



# DNS Name Resolution: Iterative Query



A host wants to resolve a hostname into an IP address

1. It sends a DNS query to its local DNS server (NYU's in this example)
2. The local DNS server forwards the query to a root DNS server
3. The root replies with the addresses of TLD (.edu) servers
4. The local DNS server queries a TLD server, which replies with the address of the authoritative DNS server for the domain (umass.edu)
5. The local DNS server then queries the authoritative server, which responds with the actual IP address of the requested host

- Total = 8 DNS messages (4 queries + 4 replies) for a single hostname resolution
- Caching is used in practice to reduce this message overhead

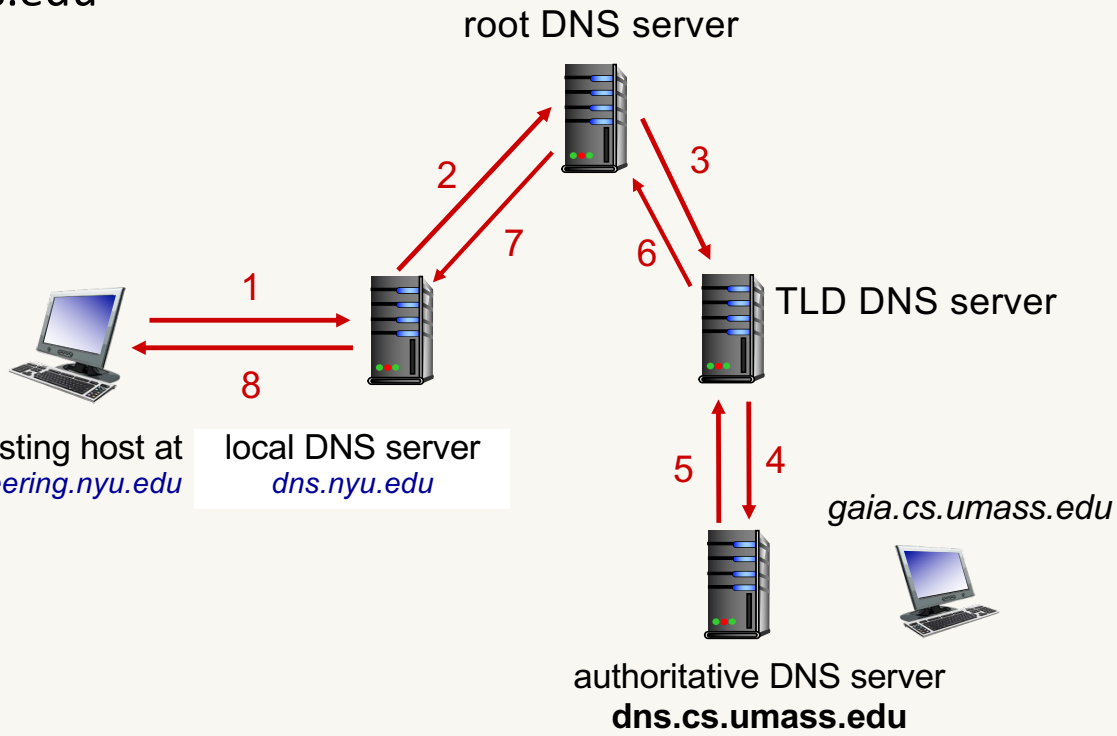
# DNS Name Resolution: Recursive Query



**Example:** host at engineering.nyu.edu wants IP address for gaia.cs.umass.edu

## Recursive query:

- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy?



# DNS Caching



- **DNS caching** is essential for reducing delay and lowering DNS query traffic
- When a **DNS server** receives a reply (hostname ⟺ IP mapping), it stores the mapping locally
- If another query for the same hostname arrives, the **DNS server** can answer directly from its **cache**, even if it's not authoritative
- Cached entries are temporary and expire after a set time (often ~2 days)
- Example: if host **A** queries for a hostname, and later host **B** queries the same hostname, the local DNS server can answer immediately from cache
- **Local DNS** servers also cache **TLD server addresses**, so they can often skip contacting **root servers**
- Result: **root** DNS servers are involved in only a small fraction of queries

# DNS Records



- DNS servers store Resource Records **(RRs)**, which are returned in DNS replies
- A **resource record** is a 4-tuple: *(Name, Value, Type, TTL)*
- **TTL (time-to-live):** specifies when a cached record should expire
- **Types of Resource Records:**
  - **Type A:** maps a hostname (*Name*) to its IP address (Value) (e.g., relay1.bar.foo.com → 145.37.93.126)
    - Example: (web.example.com, 192.0.2.10, A, 3600)
  - **Type NS:** maps a domain (*Name*) (e.g., foo.com) to the hostname (*Value*) of its authoritative DNS server
    - Example: (example.com, ns1.example.com, NS, 86400)
  - **Type CNAME:** maps an alias hostname (*Name*) to a canonical hostname (*Value*)
    - Example: (www.example.com, web.example.com, CNAME, 3600)
  - **Type MX:** maps a domain (*Name*) to the canonical name (*Value*) of a mail server (supports aliases for email servers)
    - Example: (example.com, mail.example.com, MX, 3600)

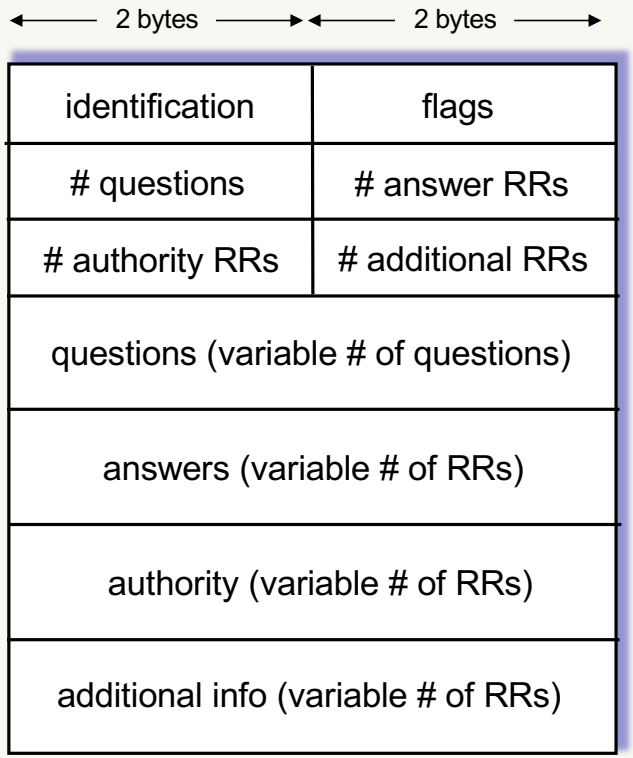
# DNS Messages



DNS **query** and **reply** messages have same format:

message header:

- **identification**: 16 bit # for query, reply to query uses same #
- **flags:**
  - query or reply
  - recursion desired
  - recursion available
  - reply is authoritative

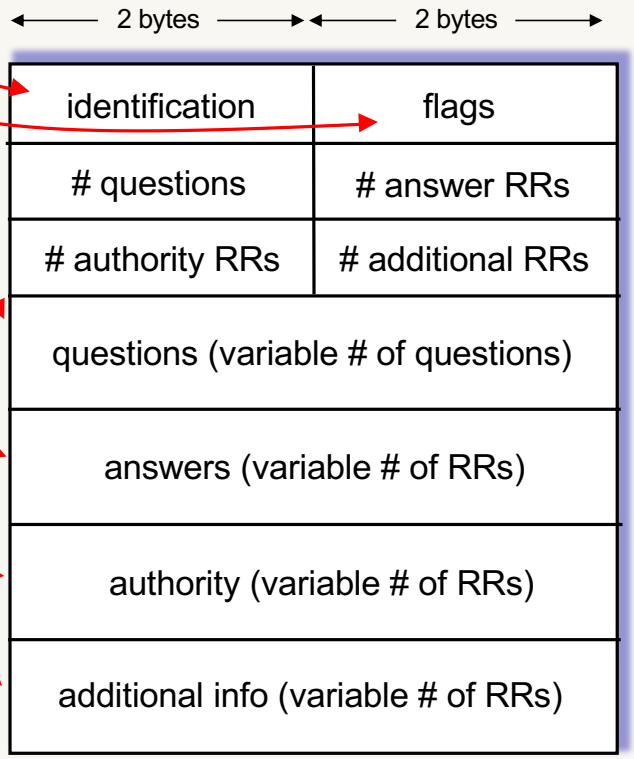


# DNS Messages



DNS **query** and **reply** messages, both have same format:

- **Header section:**
  - **Identification number**: matches replies to queries
  - **Flags**: query/reply, authoritative server, recursion desired, recursion available
  - **Counters**: number of entries in each section that follows
- **Question section:**
  - Contains the queried name
  - Contains the type of query (e.g., Type A = IP address, Type MX = mail server)
- **Answer section:**
  - Holds resource records (RRs) for the queried name
  - May return multiple RRs if a hostname maps to multiple IPs
- **Authority section:**
  - Provides details of other authoritative servers
- **Additional section:**
  - Supplies extra helpful records (e.g., IP address of a mail server when replying to an MX query)
  - Practical note: DNS queries can be made manually with the nslookup tool or through web-based nslookup services



# DNS Messages



Try: **`nslookup www.google.com`**

```
% nslookup www.google.com

Server: 131.104.20.10
Address: 131.104.20.10#53

Non-authoritative answer:
Name: www.google.com
Address: 142.251.32.68
```

- This shows the **DNS server** the machine asked to resolve the query
- #53 means the query was sent to **port 53**, the standard port for DNS

- That server returned (could be cached) **non-authoritative answer**: 142.251.32.68

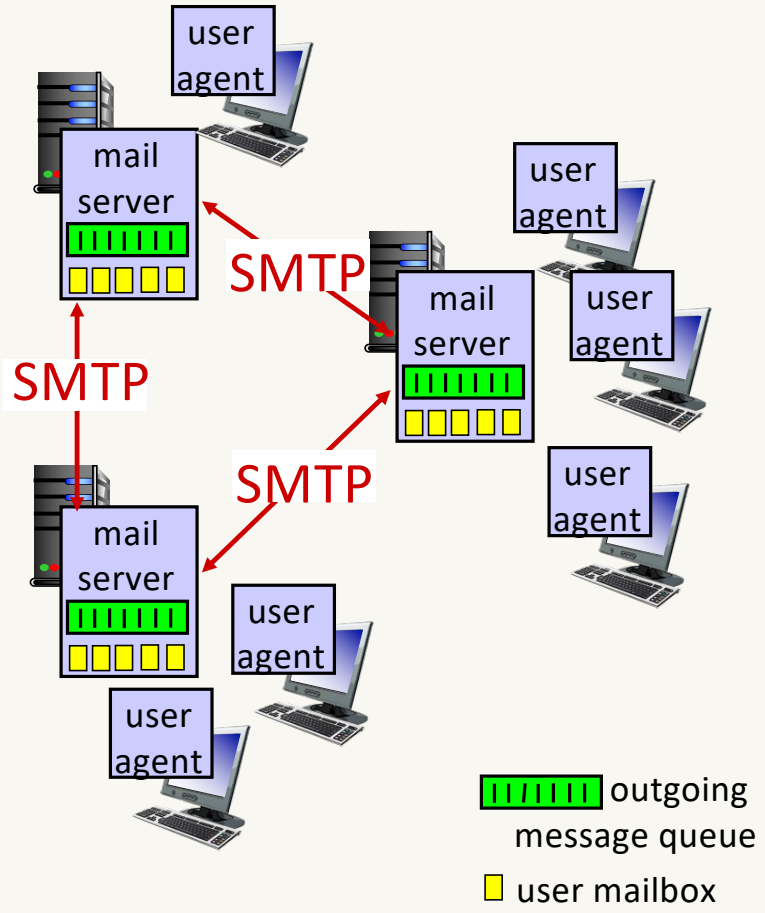
# E-mail

**Three major components:**

- user agents: compose/read/reply/forward/save
- mail servers
- simple mail transfer protocol: SMTP

**User Agent**

- a.k.a. “mail reader”
- composing, editing, reading mail messages
- e.g., Outlook, iPhone mail client
- outgoing, incoming messages stored on server



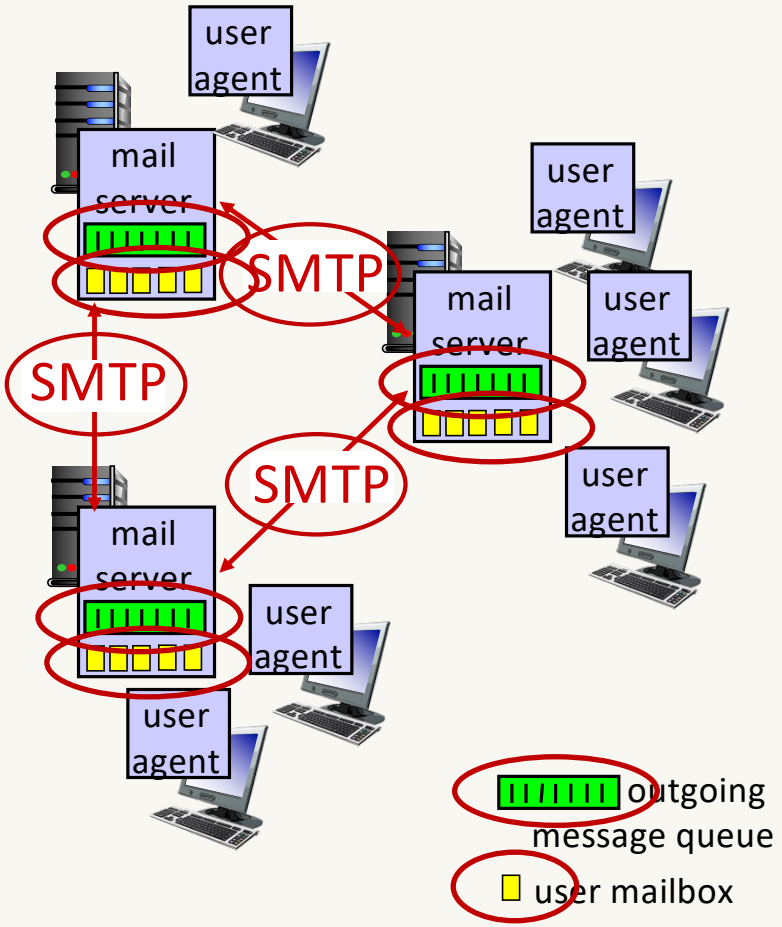
# E-mail: Mail Server

**mail servers:**

- ***mailbox*** contains incoming messages for user
- ***message queue* of** outgoing (to be sent) mail messages

**SMTP protocol** between mail servers to send email messages

- **client**: sending mail server
- **"server"**: receiving mail server





## Discussion Question:

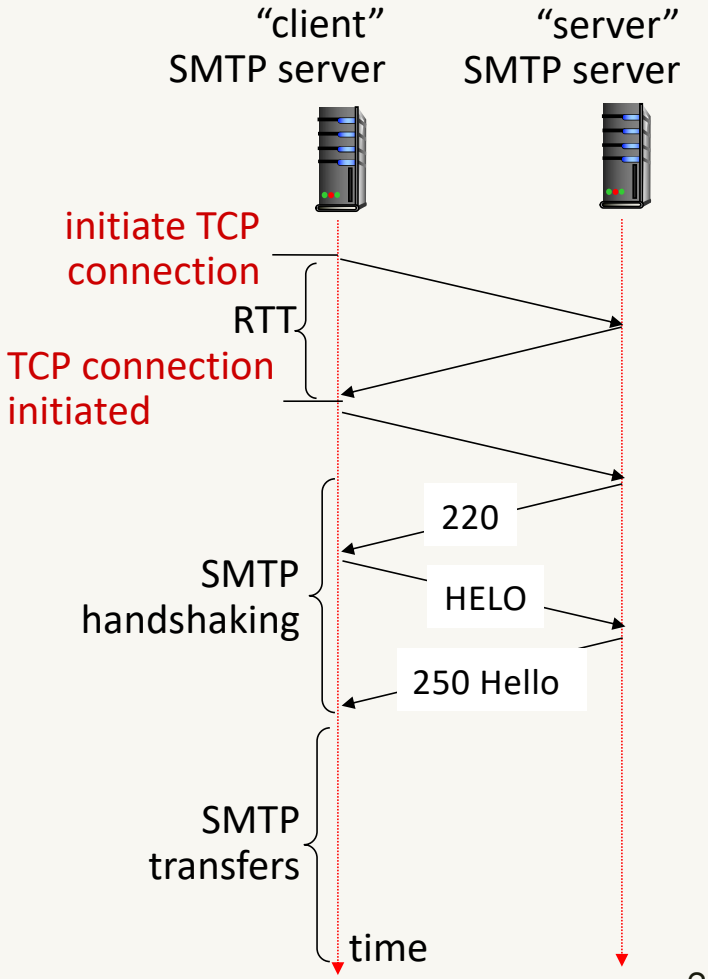
**In an SMTP exchange between two mail servers, why is the sending server called the client even though it is also a server in the email architecture?**

Client and server labels describe the role in a particular connection. The sending mail server initiates the SMTP connection, so it is the SMTP client for that exchange.

# SMTP

- Uses TCP to reliably transfer email message from client (mail server initiating connection) to server, port 25
  - direct transfer: sending server (acting like client) to receiving server
- Three phases of transfer
  - SMTP handshaking (greeting)
  - SMTP transfer of messages
  - SMTP closure
- Command/response interaction (like HTTP)
  - **commands**: ASCII text
  - **response**: status code and phrase

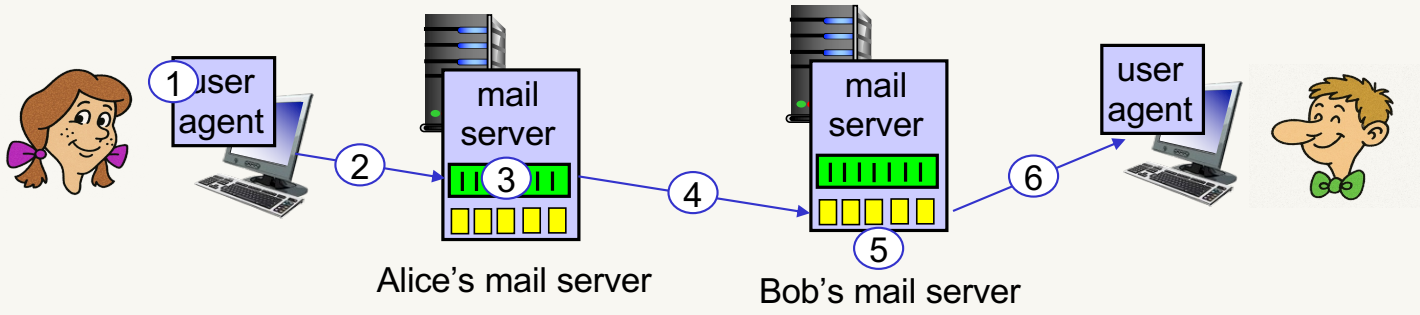




# SMTP



1. Alice uses her **user agent** to compose and send a message to Bob (bob@someschool.edu)
2. Alice's **user agent** forwards the message to her **mail server**, where it goes into the **outgoing queue**
3. SMTP client on Alice's server opens a TCP connection to Bob's **SMTP server**
4. After handshaking, Alice's message is transferred over the connection
5. Bob's **mail server** receives it and stores it in Bob's mailbox
6. Bob invokes his **user agent** to read the message



# Sample SMTP Interaction



- **SMTP Client host is** crepes.fr and **SMTP Server host is** hamburger.edu
- Lines prefaced with C: are exactly the lines the client sends into its TCP socket, and lines prefaced with S: are exactly the lines the server sends into its TCP socket

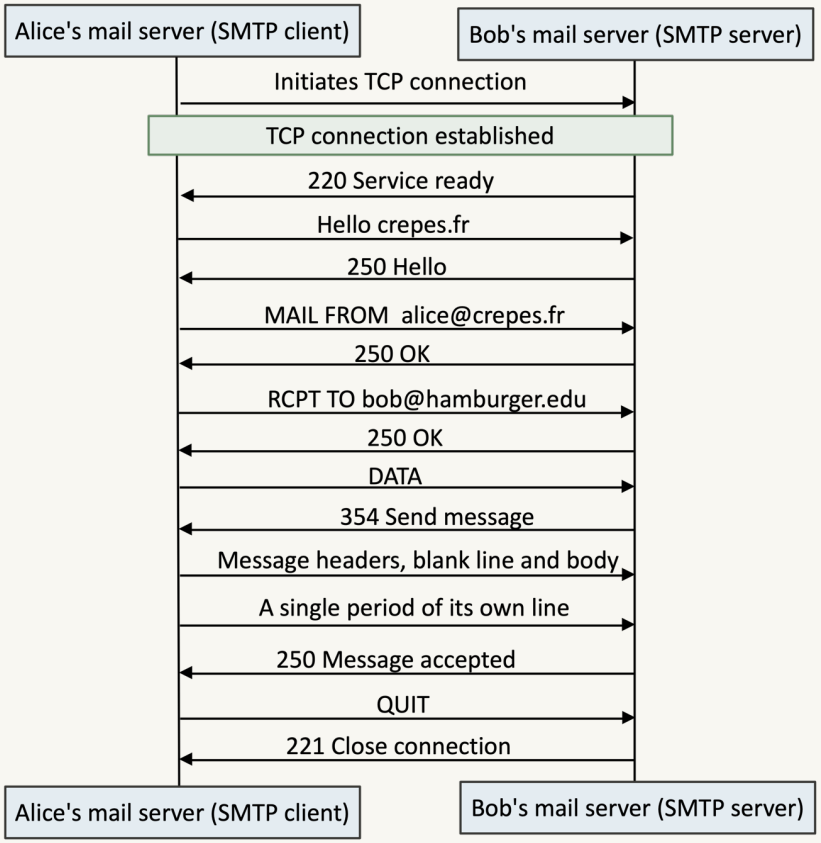
```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250  Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

Server announcing itself "Service ready"

# Sample SMTP Interaction



- **SMTP Client host is** crepes.fr and **SMTP Server host is** hamburger.edu
- Lines prefaced with C: are exactly the lines the client sends into its TCP socket, and lines prefaced with S: are exactly the lines the server sends into its TCP socket



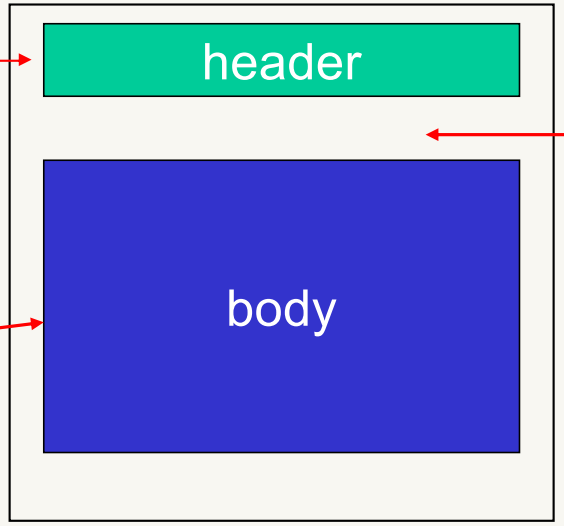
# Mail Message Format



SMTP: protocol for exchanging e-mail messages

- header lines, e.g.,
  - To:
  - From:
  - Subject:

  these lines, within the body of the email message area different from SMTP MAIL FROM:, RCPT TO: commands in the SMTP interaction!
- Body: the "message", ASCII characters only




Referring to the previous SMTP handshake, the message would be:

**Subject: Lunch preferences**
**From: alice@crepes.fr**
**To: bob@hamburger.edu**
**[blank line]**
**Do you like ketchup?**
**How about pickles?**

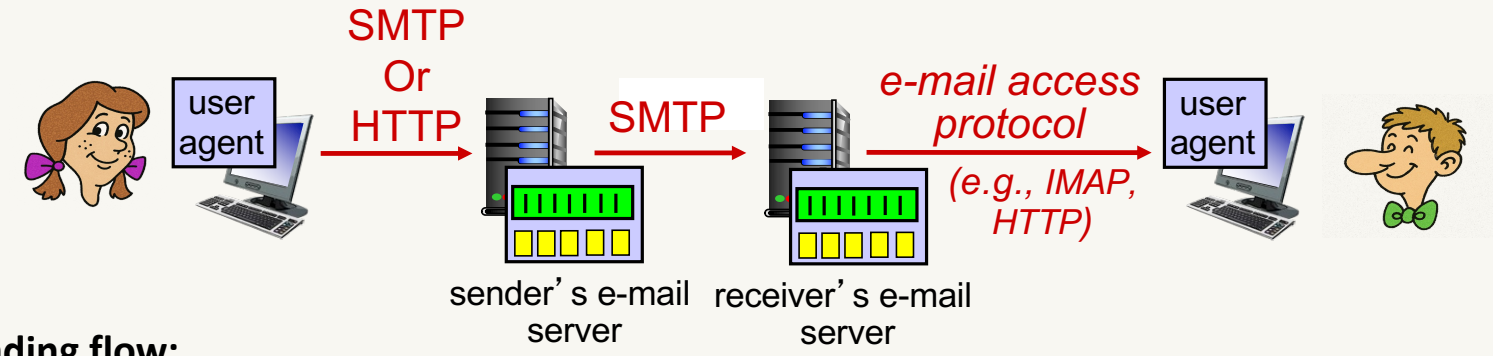
# Sample SMTP Interaction



- **Mailbox location**: Bob's messages are stored on an always-on shared mail server, not on his personal device
- **Reason**: If Bob's mail server ran on his PC/phone, it would need to be always online which is impractical for most users

# Sample SMTP Interaction






- **Sending flow:**
  - Alice's user agent sends her email to Alice's mail server (via SMTP or HTTP)
  - Alice's mail server (acting as SMTP client) relays the email to Bob's mail server
  - Benefit: Alice's server can retry delivery if Bob's server is temporarily unreachable
- **Retrieving mail:**
  - Webmail/smartphone apps (e.g., Gmail): Bob's user agent uses HTTP to fetch messages
  - Desktop clients (e.g., Outlook): Bob's user agent uses IMAP to fetch/manage mail
- **Message management**: HTTP and IMAP allow Bob to organize messages into folders, delete, or mark them
- **SMTP role**: strictly push protocol for server-to-server transfer; not used by Bob to retrieve mail

# References



- Ch. 2, Kurose and Ross



# Questions?